High-Speed Computer Arithmetic IPs: Pipelined Radix-4 Booth & Fractal Approximate Multipliers

Injected Carry-In Optimization, Brent-Kung Parallel Prefix Adders, 45 nm Spectre PVT Characterization, and Interview Defense Guide

---

Anirudh Chaturvedi

Department of Electronic Systems Engineering (DESE)
Indian Institute of Science (IISc), Bangalore

Implementation: AMD Xilinx Artix-7 FPGA & Cadence Virtuoso 45 nm CMOS

Abstract

This monograph covers the mathematical derivation, microarchitecture, and physical characterization of two high-performance computer arithmetic IPs: (1) A signed 8-bit pipelined Radix-4 Booth recoded multiplier utilizing an injected carry-in optimization, Carry-Save reduction trees, and a 16-bit Brent-Kung parallel prefix adder achieving 165 MHz physical timing closure on Artix-7 (270.2 MHz theoretical $F_{\max}$); and (2) An 8-bit configurable approximate multiplier featuring Fractal Partial-Product Pruning (LFM/SFM) characterized in Cadence Spectre across PVT corners, achieving an **18.0% dynamic power reduction** and an **18.0% delay cut**.

Contents

1 Stage 1: Signed 8-bit Radix-4 Booth Multiplier **2**
1.1 Radix-4 Overlapping Triplet Recoding 2
1.2 The Injected Carry-In Optimization 2
1.3 16-Bit Brent-Kung Parallel Prefix Adder 2

2 Stage 2: Configurable Low-Power Approximate Multiplier **3**
2.1 Fractal Partial-Product Pruning (LFM / SFM) 3
2.2 Cadence Spectre PVT Characterization (45 nm) 3

3 Stage 3: Interview Defense Dialogues **4**

Chapter 1

Stage 1: Signed 8-bit Radix-4 Booth Multiplier

1.1 Radix-4 Overlapping Triplet Recoding

For signed multiplication $P = X \times Y$, Radix-4 Booth recoding inspects overlapping 3-bit triplets of the multiplier $(y_{2i+1}, y_{2i}, y_{2i-1})$ with $y_{-1} = 0$, reducing partial product rows from 8 to 4:

$$PP_i = \text{BoothRecode}(y_{2i+1}, y_{2i}, y_{2i-1}) \cdot X \cdot 4^i \tag{1.1}$$

Multiplication operations are restricted to $\{0, \pm X, \pm 2X\}$.

1.2 The Injected Carry-In Optimization

Negating a partial product in two's complement requires inverting bits and adding 1:

$$-A = \sim A + 1 \tag{1.2}$$

Instead of dedicating adder stages to perform this addition, the +1 carry-in bit is directly injected into the vacant least-significant bit position of the downstream Carry-Save Adder (CSA) reduction tree, achieving sign inversion with **zero additional logic gates and zero propagation delay**.

1.3 16-Bit Brent-Kung Parallel Prefix Adder

The compressed sum (S) and carry (C) vectors are assimilated using a 16-bit Brent-Kung parallel prefix tree. With a tree depth of $2\log_2 N - 1 = 7$ logic stages, Brent-Kung achieves minimal fanout and highly regular routing compared to Kogge-Stone, minimizing routing area.

Chapter 2

Stage 2: Configurable Low-Power Approximate Multiplier

2.1 Fractal Partial-Product Pruning (LFM / SFM)

In fault-tolerant DSP and machine learning inference, least-significant partial products contribute minimally to final output precision. We formulated two pruning rules:

- **Light Fractal Mode (LFM):** Truncates the bottom 4 triangular partial-product bits.
- **Strong Fractal Mode (SFM):** Prunes the bottom 8 partial-product bits using fractal thresholding.

2.2 Cadence Spectre PVT Characterization (45 nm)

Characterized across Fast-Fast, Typical-Typical, and Slow-Slow process corners with 2 fF load:

- **Dynamic Power:** Reduced from 182.8 μW to 149.9 μW (**18.0% power reduction**).
- **Critical Path Delay:** Reduced from 254.4 ps to 208.6 ps (**18.0% delay cut**).
- **Mean Relative Error (MRED):** Kept below 1.8% across image filtering benchmarks.

Chapter 3

Stage 3: Interview Defense Dialogues

Q1: Why choose Radix-4 Booth recoding over Radix-8 for an 8-bit multiplier?

Candidate Defense: Radix-4 Booth recoding requires generating only $\pm X$ and $\pm 2X$, where $2X$ is a simple 1-bit hardwired left shift. In contrast, Radix-8 Booth recoding requires generating $\pm 3X$, which is a non-power-of-two multiple requiring a dedicated carry-propagate adder before multiplication begins. For an 8-bit multiplier, the latency and area overhead of generating $3X$ completely outweighs the benefit of reducing partial products from 4 to 3.

Q2: How do Brent-Kung and Kogge-Stone adders compare in ASIC physical layout?

Candidate Defense: Kogge-Stone adders have the lowest logical depth ($\log_2 N = 4$ stages for 16 bits), but require massive wiring density with high fanout at higher stages, causing severe routing congestion and interconnect RC delay in deep-submicron nodes. Brent-Kung adders trade logical depth ($2\log_2 N - 1 = 7$ stages) for strictly bounded fanout (≤ 2) and tree-like wire tracks. In sub-45 nm standard-cell layouts, Brent-Kung frequently achieves smaller total area and lower power with comparable delay once parasitic interconnect capacitance is extracted.